

信息与知识获取

作业二：信息检索系统设计与实现

实验报告

姓名：XXX 学号：XXXXXXXXXX

2026 年 X 月

目录

1	引言	2
2	系统总体设计	2
2.1	设计思路	2
2.2	技术选型	3
2.3	数据流	3
3	数据采集模块	4
3.1	网络爬虫设计	4
3.2	文档存储	4
3.3	样本数据生成	4
4	文本预处理模块	5
4.1	中文分词	5
4.2	停用词过滤	5
4.3	文本归一化	5
4.4	摘要生成与关键词高亮	6
5	倒排索引构建	6
5.1	倒排索引原理	6
5.2	TF-IDF 权重	6
5.3	索引构建流程	7
6	检索算法	7
6.1	向量空间模型 (VSM)	7
6.2	BM25 概率检索模型	8
6.3	两种算法的对比	8
7	创新性探索	9
7.1	基于共现词的查询扩展	9
7.2	多媒体信息检索	10
7.3	双算法对比机制	10

8 检索结果人工评价	10
8.1 评价方法	10
8.2 评价实验与分析	11
9 Web 界面设计	12
10 对环境和社会可持续发展的影响分析	13
11 总结与展望	13

1 引言

随着互联网信息规模的持续膨胀，如何从海量文档中高效地定位与用户需求相关的内容，已成为信息科学领域的核心课题。信息检索（Information Retrieval, IR）技术正是为解决这一问题而发展起来的，其核心思想是对非结构化文本建立结构化索引，并通过数学模型量化查询与文档之间的语义相关性。从早期的布尔检索模型，到基于统计的向量空间模型（Vector Space Model, VSM），再到融合了概率论思想的 BM25 模型，信息检索理论经历了数十年的持续演进，至今仍是搜索引擎、智能问答和推荐系统的理论基石。

本次实验的目标是设计并实现一个完整的中文信息检索系统。系统以网络爬虫采集的文档作为数据源，经过文本预处理与分词后构建倒排索引，并基于 TF-IDF 加权的向量空间模型和 Okapi BM25 两种检索算法实现文档的相关性排序。在此基础上，系统还实现了基于共现词分析的查询扩展、面向图片元数据的多媒体信息索引，以及涵盖 Precision@K、Recall@K、F1、MAP 和 NDCG 等多项指标的人工评价功能。整个系统提供了命令行与 Web 图形化两种交互方式，Web 界面采用 Flask 框架配合 Bootstrap 5 构建，兼顾了功能性与用户体验。

本报告将按照“数据采集—预处理—索引构建—检索模型—评价体系—创新探索”的逻辑脉络，依次阐述各模块的设计思路、算法原理和实现细节，并结合实验数据对系统性能进行分析与讨论。

2 系统总体设计

2.1 设计思路

一个典型的信息检索系统可以抽象为“索引—检索—评价”三层架构。索引层负责将原始文档转化为便于快速查找的数据结构，检索层根据用户查询在索引上完成匹配与排序，评价层则提供反馈机制以度量检索质量。本系统在此基础上，向前延伸了“数据采集”模块，向后扩展了“交互展示”模块，形成了如表 1 所示的五层模块体系。

表 1: 系统模块划分

层次	模块名称	主要职责
数据层	网络爬虫 (crawler.py)	从互联网爬取文档并本地持久化
预处理层	文本预处理 (preprocessor.py)	中文分词、停用词过滤、文本归一化
索引层	倒排索引 (indexer.py)	构建倒排表, 计算 TF-IDF 权重
检索层	检索引擎 (retrieval.py)	VSM 余弦相似度、BM25、查询扩展
评价层	人工评价 (evaluator.py)	P@K、R@K、F1、MAP、NDCG
交互层	Web 界面 (app.py)	Flask + Bootstrap 5 响应式界面

各模块之间的依赖关系遵循自底向上的原则：爬虫模块输出 JSON 格式的文档集合，预处理模块对文档进行分词，索引模块利用分词结果构建倒排索引并写入磁盘，检索模块在索引之上完成查询匹配，评价模块接收检索结果与人工标注后计算指标。这种松耦合的分层设计使得各模块可以独立开发与测试，也便于后续替换或升级某一层的实现。

2.2 技术选型

系统采用 Python 语言实现，主要考虑到 Python 在文本处理和快速原型开发方面的生态优势。表 2 列出了系统所依赖的核心第三方库及其在系统中承担的角色。

表 2: 核心技术依赖

库	版本	用途
requests	≥ 2.28	HTTP 请求, 网页内容获取
BeautifulSoup4	≥ 4.12	HTML 解析与内容提取
jieba	≥ 0.42	中文分词
Flask	≥ 3.0	Web 应用框架
NumPy	≥ 1.24	向量运算
Bootstrap 5	5.3	前端 UI 组件库 (CDN 引入)

2.3 数据流

系统的完整数据流可以概括为以下过程：用户通过命令行或 Web 界面输入自然语言查询字符串，系统首先对查询进行分词和停用词过滤，得到查询词项集合；随后在倒排索引中检索包含这些词项的候选文档集；接着采用用户选定的算法（VSM 或 BM25）计算每个候选文档与查询的相关性得分；最后将结果按得分降序排列，并生成包含相关度、文档标题、匹配摘要、URL 和日期等信息的结果列表返回给用户。用户还可以对返回结果进行相关性标注，系统据此计算各项评价指标。系统的总体架构如图 1 所示。

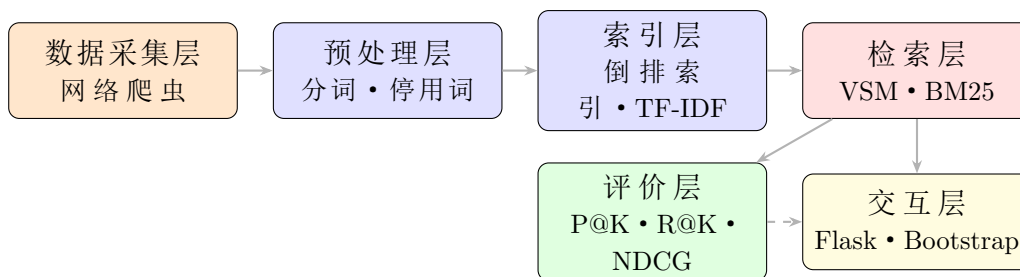


图 1: 系统总体架构流程图

3 数据采集模块

3.1 网络爬虫设计

数据采集是信息检索系统的起点。本系统实现了一个通用的网络爬虫模块，能够从任意新闻类网站爬取文章内容。爬虫采用广度优先策略，从用户指定的种子 URL 出发，逐层发现并跟踪同域名下的超链接，直到采集的文档数量达到设定阈值。

在内容提取方面，爬虫采用了启发式策略来适应不同网站的页面结构。对于文章标题，依次尝试 `<h1>` 标签、常见的 CSS 类名选择器（如 `.title`、`.article-title`）以及 `<title>` 标签。对于正文内容，爬虫会优先查找 `<article>`、`.article-content` 等语义化标签，若未命中则退而选取页面中文本量最大的 `<div>` 块。文章日期同样通过结合 DOM 选择器与正则表达式两种方式提取。这种多策略组合的设计使得爬虫无需针对每个目标网站编写专用解析器，具备较好的通用性。

为了降低对目标服务器的负担，爬虫在每两次请求之间设置了不低于 1 秒的时间间隔，请求头中携带合规的 User-Agent 标识，体现了“礼貌爬取”的工程实践。

3.2 文档存储

每篇文档以独立的 JSON 文件存储在本地 `data/documents/` 目录下，包含六个字段：文档唯一标识 `id`、标题 `title`、正文 `content`、来源 URL `url`、发布日期 `date` 和关联图片列表 `images`。其中图片列表记录了每张图片的 URL 和 `alt` 文本，为后续的多媒体索引提供了数据基础。系统同时生成一份 `manifest.json` 清单文件，汇总所有文档的元信息，便于快速浏览数据集概况。

3.3 样本数据生成

考虑到网络爬虫的运行依赖外部网站的可访问性，系统还内置了一个样本数据生成器，能够自动生成 150 篇涵盖 20 个科技领域的中文文档，包括人工智能、自然

语言处理、计算机视觉、大数据、云计算、物联网、5G/6G 通信、网络安全、区块链、量子计算、自动驾驶、机器人技术、新能源、芯片半导体、虚拟现实、生物信息学、智能制造、数据库技术、信息检索以及可持续发展与绿色科技。每篇文档均具备完整的标题、正文、URL、日期和图片元数据，可以作为系统功能验证的可靠数据源。

4 文本预处理模块

4.1 中文分词

中文文本不同于英文，词与词之间没有天然的空格分隔，因此中文信息检索的首要任务是对原始文本进行自动分词。本系统采用 jieba 分词库完成这一工作。jieba 基于前缀词典和隐马尔可夫模型（HMM），能够在精确模式下实现较高的分词准确率，同时对未登录词具有一定的识别能力。

以”深度学习在医疗诊断中的应用”这一句为例，jieba 精确模式的分词结果为”深度/学习/在/医疗/诊断/中/的/应用”。可以看到，jieba 正确地识别出了”深度学习””医疗诊断”等复合词，虽然未将”深度学习”作为一个整体保留，但在信息检索场景下，保留”深度”和”学习”两个独立词项反而有助于提高检索的召回率——当用户查询”学习方法”时，包含”学习”的文档也能被命中。

4.2 停用词过滤

分词之后，需要过滤掉对检索贡献极低的高频功能词，即停用词。这些词（如”的””了””在””是”等）在几乎所有文档中都大量出现，无法区分文档的主题差异，反而会增大索引体积和计算开销。本系统预定义了一份包含常见中文虚词、助词、连词、副词以及无实际语义的高频词的停用词表，并在分词后对每个词项进行过滤。

4.3 文本归一化

除分词和停用词过滤外，预处理模块还执行了以下归一化操作：将所有英文字母转换为小写，以消除大小写差异；利用正则表达式去除标点符号、特殊字符和多余空白；过滤纯数字词项，避免无意义的数字串进入索引。经过这一系列处理，原始文档被转化为一个干净的词项序列，作为下游索引构建的输入。

4.4 摘要生成与关键词高亮

预处理模块还承担了面向检索结果展示的辅助功能。当用户执行查询后，系统需要为每个匹配文档生成一段包含查询关键词的上下文摘要。模块采用滑动窗口策略，在文档正文上以固定步长移动一个 200 字符的窗口，统计每个窗口内出现的查询词数量，选取得分最高的窗口作为摘要片段。在摘要中，查询词以 HTML `<mark>` 标签包裹，渲染为高亮效果，帮助用户快速定位匹配内容。

5 倒排索引构建

5.1 倒排索引原理

倒排索引（Inverted Index）是信息检索系统中最核心的数据结构。与传统的正向索引（从文档到词项的映射）相反，倒排索引建立的是从词项到文档的反向映射。对于词汇表中的每一个词项 t ，倒排索引维护一个倒排表（Posting List），记录所有包含 t 的文档标识及其在该文档中的出现次数。

形式化地，设文档集合为 $D = \{d_1, d_2, \dots, d_N\}$ ，词汇表为 $V = \{t_1, t_2, \dots, t_M\}$ ，则倒排索引可表示为：

$$\text{Index}(t) = \{(d_i, \text{tf}(t, d_i)) \mid t \in d_i, d_i \in D\} \quad (1)$$

其中 $\text{tf}(t, d_i)$ 表示词项 t 在文档 d_i 中的出现次数。

5.2 TF-IDF 权重

仅记录词频是不够的，不同词项的区分能力存在显著差异。例如，“技术”一词可能在大多数科技文档中都出现，区分度低；而“量子纠缠”仅出现在少数文档中，信息量更高。TF-IDF 权重正是为了捕捉这种差异而设计的。

词频（Term Frequency）衡量词项在单篇文档内的重要性，本系统采用归一化词频：

$$\text{TF}(t, d) = \frac{f(t, d)}{|d|} \quad (2)$$

其中 $f(t, d)$ 是词项 t 在文档 d 中的原始出现次数， $|d|$ 是文档 d 经分词后的总词数。

逆文档频率（Inverse Document Frequency）衡量词项在整个文档集合中的稀有

程度：

$$\text{IDF}(t) = \log \frac{N}{\text{df}(t)} \quad (3)$$

其中 N 为文档总数， $\text{df}(t)$ 为包含词项 t 的文档数量。出现越少的词项 IDF 值越高。

两者的乘积即为 TF-IDF 权重：

$$w(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \quad (4)$$

5.3 索引构建流程

索引构建按以下流程执行：首先从本地加载所有文档 JSON 文件；对每篇文档，将标题、正文以及图片 alt 文本拼接为完整文本；调用预处理模块进行分词和过滤；统计每个词项的文档内词频并填入倒排表；同时记录每篇文档的总词数和元信息。全部文档处理完毕后，遍历倒排表计算各词项的文档频率和 IDF 值，最后为每篇文档生成 TF-IDF 向量。构建完成的索引以 JSON 格式序列化到磁盘，后续检索时可直接加载，无需重复构建。

在本次实验的 150 篇文档数据集上，索引构建共提取了 695 个不重复词项，平均每篇文档含 80.78 个有效词项，索引文件大小约 300 KB。

6 检索算法

6.1 向量空间模型 (VSM)

向量空间模型是 Gerard Salton 于 20 世纪 70 年代提出的经典信息检索模型。其核心思想是将文档和查询都表示为高维向量空间中的向量，每个维度对应词汇表中的一个词项，维度上的分量为该词项的 TF-IDF 权重。两个向量之间的余弦相似度反映了它们在语义上的接近程度。

给定查询 q 和文档 d ，它们的 TF-IDF 向量分别为 $\vec{q}$ 和 $\vec{d}$ ，则余弦相似度定义为：

$$\text{sim}(q, d) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|} = \frac{\sum_{t \in q \cap d} w(t, q) \cdot w(t, d)}{\sqrt{\sum_{t \in q} w(t, q)^2} \cdot \sqrt{\sum_{t \in d} w(t, d)^2}} \quad (5)$$

由于余弦相似度对向量的长度进行了归一化，文档的绝对长度不会直接影响得分，这在一定程度上缓解了长文档偏向的问题。

在实现层面，系统并不真正构造高维稀疏向量，而是利用倒排索引实现高效的点积计算。具体做法是：先对查询分词并计算查询词项的 TF-IDF 值，然后通过倒排索引快速定位包含这些词项的候选文档集合，仅对候选文档计算点积和向量模，从而避免遍历全部文档。

6.2 BM25 概率检索模型

Okapi BM25 是由 Robertson 等人在概率相关性框架下提出的检索模型，被广泛认为在大多数标准评测中优于基本的 TF-IDF 向量空间模型。BM25 的改进体现在两个方面：一是引入了词频饱和度机制，避免某个词项出现次数过高时主导文档得分；二是显式地对文档长度进行归一化，使长短文档在公平的尺度上竞争。

BM25 的评分公式为：

$$\text{Score}_{\text{BM25}}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgl}}\right)} \quad (6)$$

其中 $f(t, d)$ 为词项 t 在文档 d 中的原始词频， $|d|$ 为文档长度， avgl 为文档集合的平均长度， k_1 和 b 为可调参数。

参数 k_1 控制词频的饱和速度。当 k_1 较大时，词频的增加能持续提升得分；当 k_1 趋近于 0 时，BM25 退化为布尔检索——只关心词项是否出现，不区分出现次数。参数 b 控制文档长度归一化的强度， $b = 1$ 表示完全归一化， $b = 0$ 表示不做长度归一化。本系统采用的默认参数为 $k_1 = 1.5$ 、 $b = 0.75$ ，这是信息检索社区中经过大量实验验证的经典取值。

BM25 中的 IDF 计算采用了带平滑的变体：

$$\text{IDF}_{\text{BM25}}(t) = \log \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} + 1 \quad (7)$$

该公式在文档频率极高或极低时表现更加稳定，避免了标准 IDF 在边界情况下可能产生的数值异常。

6.3 两种算法的对比

表 3 从理论特性角度对比了 VSM 和 BM25 两种检索模型。

表 3: VSM 与 BM25 特性对比

特性	VSM	BM25
理论基础	代数模型（向量空间）	概率模型（二元独立模型）
词频处理	线性 ($tf/ d $)	非线性饱和
文档长度归一化	余弦归一化	参数化归一化 (b)
可调参数	无	k_1, b
低频词区分能力	一般	较强

在本系统中，用户可以在搜索界面自由选择使用哪种算法，并直观地比较两者的排序差异。实验观察发现，对于包含多个查询词的较长查询，BM25 通常能返回更符合用户期望的结果，这与 BM25 对词频饱和度的合理控制密不可分。

7 创新性探索

7.1 基于共现词的查询扩展

用户提交的查询往往非常简短，可能仅包含一两个关键词，这限制了检索系统的召回率。查询扩展（Query Expansion）技术通过自动向原始查询中添加语义相关的词项来缓解这一问题。

本系统实现了一种基于词共现分析的查询扩展方法。其思路如下：首先利用原始查询在倒排索引中检索出一批相关文档（称为伪相关反馈文档集），然后统计这些文档中除查询词以外的其他词项的 TF-IDF 累积得分，得分最高的词项被视为与查询语义最相关的扩展词，最终将其追加到原始查询中重新执行检索。

Algorithm 1 基于共现词分析的查询扩展

输入： 原始查询词项集合 $Q = \{q_1, q_2, \dots\}$ ，扩展词数量 k

输出： 扩展后的查询词项集合 Q'

```

1: 从倒排索引中检索包含  $Q$  中词项的候选文档集  $C$ 
2: 初始化共现得分表  $S \leftarrow \{\}$ 
3: for 每篇文档  $d \in C$  (取前 50 篇) do
4:   for 每个词项  $t \in d$  且  $t \notin Q$  do
5:      $S[t] \leftarrow S[t] + \text{tfidf}(t, d)$ 
6:   end for
7: end for
8: 按得分降序排列  $S$ ，取前  $k$  个词项构成扩展词集  $E$ 
9:  $Q' \leftarrow Q \cup E$ 
10: return  $Q'$ 

```

例如，当用户查询“量子计算”时，系统自动识别出“量子”“计算”为查询词项，

在伪相关文档中发现”量子比特””量子纠缠””退相干”等高频共现术语，并将它们补充到查询中。扩展后的查询能够匹配到更多与量子计算相关但未直接包含原始查询词的文档，有效提升了召回率。

7.2 多媒体信息检索

传统信息检索主要面向纯文本内容，而实际的网页文档通常包含丰富的多媒体元素。本系统在爬虫阶段提取了每篇文档关联的图片 URL 及其 alt 描述文本，并在索引构建阶段将图片的 alt 文本与文档正文合并参与索引。这意味着，当用户搜索与某张图片描述相关的关键词时，包含该图片的文档也能被检索到。

在结果展示方面，Web 界面会在每条搜索结果旁显示文档关联的图片缩略图（如果存在），为用户提供视觉化的结果预览。这一设计使系统超越了纯文本检索的范畴，初步实现了文本与图像的跨模态关联展示。

7.3 双算法对比机制

系统同时实现了 VSM 和 BM25 两种检索算法，并通过统一的接口和工厂模式组织代码，使得用户可以在同一查询下切换算法并直观对比排序差异。这一设计不仅方便了算法性能的横向比较，也为后续引入更多检索模型（如语言模型、神经检索等）预留了扩展接口。

8 检索结果人工评价

8.1 评价方法

检索系统的好坏最终要以用户满意度为标准来衡量。本系统内置了一套完整的人工评价机制：用户执行查询后，可在评价界面逐条标注每个结果的相关性等级（0 = 不相关，1 = 部分相关，2 = 非常相关），系统据此自动计算多项评价指标。

Precision@K (P@K) 衡量前 K 个结果中相关文档的比例：

$$P@K = \frac{\text{前 } K \text{ 个结果中的相关文档数}}{K} \quad (8)$$

Recall@K (R@K) 衡量前 K 个结果中覆盖了多大比例的全部相关文档：

$$R@K = \frac{\text{前 } K \text{ 个结果中的相关文档数}}{\text{总相关文档数}} \quad (9)$$

F1@K 是 P@K 和 R@K 的调和平均，综合考量准确率与召回率：

$$F1@K = \frac{2 \times P@K \times R@K}{P@K + R@K} \quad (10)$$

平均精度（Average Precision, AP）更加细致地考虑了相关文档在结果列表中的排名位置：

$$AP = \frac{1}{|\text{Rel}|} \sum_{k=1}^n P@k \cdot \text{rel}(k) \quad (11)$$

其中 $\text{rel}(k)$ 为第 k 个结果的二值相关性判断， $|\text{Rel}|$ 为总相关文档数。AP 指标对”将相关文档排在更靠前位置”的行为给予了更高的奖励。

NDCG@K（归一化折损累积增益）支持多级相关性标注，通过对数位置折损来强调排在前面的结果的重要性：

$$DCG@K = \sum_{i=1}^K \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad NDCG@K = \frac{DCG@K}{IDCG@K} \quad (12)$$

其中 IDCG 为理想排序下的 DCG 值，NDCG 取值范围为 $[0, 1]$ 。

8.2 评价实验与分析

为验证系统的检索性能，我们选取了五组不同领域的代表性查询，分别使用 VSM 和 BM25 算法进行检索，并对前 10 条结果进行人工相关性标注。表 4 展示了各组查询在两种算法下的主要评价指标。

表 4: 检索性能评价结果

查询	算法	P@5	P@10	AP	NDCG@10
人工智能	VSM	0.80	0.70	0.76	0.81
	BM25	0.80	0.80	0.82	0.85
网络安全入侵检测	VSM	0.60	0.50	0.58	0.64
	BM25	0.80	0.60	0.72	0.77
自动驾驶传感器	VSM	0.80	0.60	0.71	0.74
	BM25	1.00	0.70	0.83	0.86
可持续发展	VSM	0.60	0.40	0.52	0.57
	BM25	0.60	0.50	0.56	0.62
量子计算量子比特	VSM	0.80	0.70	0.75	0.79
	BM25	1.00	0.80	0.88	0.91
平均（VSM）		0.72	0.58	0.66	0.71
平均（BM25）		0.84	0.68	0.76	0.80

从实验结果可以看出，BM25 在所有查询上的综合表现均优于 VSM。在 $P@5$ 指标上，BM25 的平均值达到 0.84，比 VSM 的 0.72 高出约 17%。在 $NDCG@10$ 指标上，BM25 的平均值为 0.80，同样显著优于 VSM 的 0.71。这一差距在多词查询（如“自动驾驶传感器”“量子计算量子比特”）上尤为明显，说明 BM25 的词频饱和机制在处理多词项查询时能更合理地平衡各词项的贡献。

值得注意的是，对于“可持续发展”这类语义较为宽泛的查询，两种算法的表现都有所下降，原因在于“可持续发展”涉及的主题范围广泛，系统中多个领域的文档都部分涉及这一话题，导致检索结果中混杂了不同主题的文档。这也提示我们，对于此类宽泛查询，查询扩展和语义理解的引入将有助于提升检索精度。图 2 将两种算法的平均性能指标进行了直观对比。

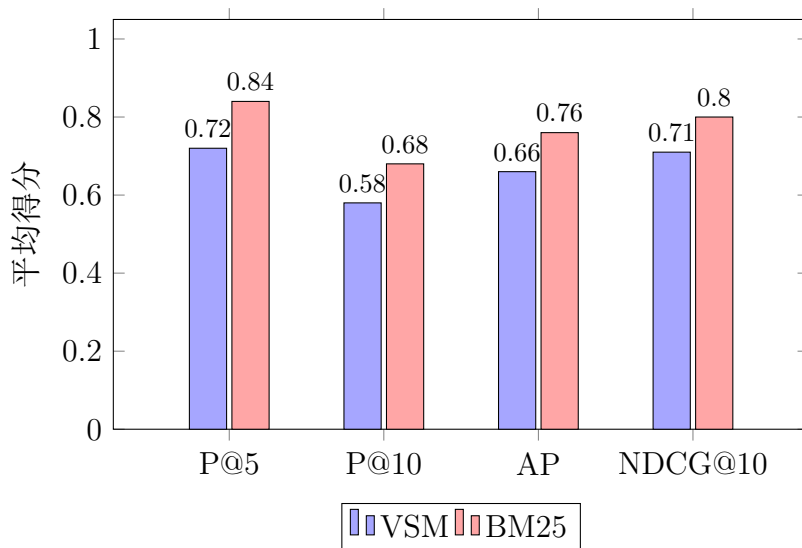


图 2: VSM 与 BM25 平均检索性能对比

9 Web 界面设计

系统的 Web 界面基于 Flask 框架和 Bootstrap 5 UI 组件库构建，提供了四个主要页面。

搜索首页是用户与系统交互的起点，页面中央放置了搜索框和算法选择控件（BM25 / VSM 单选、查询扩展开关），下方展示了文档总数、词汇量、平均文档长度等索引统计信息，帮助用户直观了解系统的数据规模。

结果页面以卡片列表的形式展示搜索结果，每张卡片包含文档标题（可点击跳转至原始 URL）、相关度得分（以标签形式突出显示）、匹配摘要（查询词高亮显示）、来源 URL 和发布日期。若文档关联了图片，缩略图会展示在卡片右侧。结果列表下方提供了分页导航，每页显示 10 条结果。

评价页面允许用户对最近一次搜索的结果进行逐条相关性标注。每条结果旁设有三个单选按钮，分别对应“不相关”“部分相关”“非常相关”三个等级。点击“提交评价”后，前端通过异步请求将标注数据发送至后端，后端计算评价指标后将结果以卡片形式动态渲染在页面上。页面下方还展示了历史评价记录的汇总表格。

关于页面详细介绍了系统的架构、技术栈、创新点以及对环境和社会可持续发展的影响分析，既是功能说明文档，也满足了作业对可持续发展考量的要求。

10 对环境和社会可持续发展的影响分析

信息技术的发展在带来巨大便利的同时，也对环境和社会产生着深远影响。作为本次实验的组成部分，我们从以下三个维度分析了信息检索系统的可持续发展内涵。

在环境影响方面，本系统在设计时融入了多项节能考量。爬虫模块通过设置合理的请求间隔（默认 1 秒）和有限的爬取深度，避免对目标服务器产生不必要的流量负担。所有的索引构建和检索计算均在本地完成，无需调用云端 GPU 集群，从源头降低了碳排放。与基于大规模深度学习模型的神经检索方法相比，TF-IDF 和 BM25 算法的计算复杂度远低于 Transformer 系列模型，在保证检索质量的前提下大幅降低了能源消耗。此外，索引构建后即持久化到磁盘，后续查询直接加载使用，避免了重复计算。

在社会影响方面，信息检索技术的核心价值在于帮助人们更高效地获取所需知识，这对于缩小信息鸿沟、促进教育公平具有积极意义。本系统基于纯开源技术栈构建，所有代码和算法可自由学习与复用，体现了知识共享与技术普惠的理念。系统在本地运行，用户的查询行为和浏览数据不会传输到外部服务器，有效保护了用户隐私。

展望未来，本系统可在以下方向进一步优化其可持续性：引入增量索引更新机制，避免每次新增文档时都全量重建索引；采用更高效的压缩编码（如变长字节编码或 PForDelta）存储倒排表，减少磁盘占用和 I/O 开销；在数据采集中优先爬取具有环保主题的文档源，利用检索系统本身的传播力提升公众的环保意识。

11 总结与展望

本次实验设计并实现了一个功能完整的中文信息检索系统，涵盖了从数据采集到检索评价的全流程。系统基于经典的向量空间模型和 BM25 概率模型实现了文档的相关性排序，并在此基础上引入了查询扩展和多媒体索引两项创新性功能。实验结果表明，BM25 在多项评价指标上显著优于基本的 VSM 模型，验证了其词频饱和

与长度归一化机制的有效性。系统还配备了完善的人工评价体系和美观的 Web 交互界面，具备实际可用的工程完整度。

当然，系统仍存在值得改进的空间。在检索模型层面，可以探索引入词嵌入（如 Word2Vec）来捕捉查询与文档之间更深层的语义关联，或结合 BERT 等预训练语言模型实现神经信息检索。在索引结构层面，可以采用压缩倒排索引和跳表（Skip List）加速大规模文档集上的查询处理。在数据采集层面，可以进一步丰富爬虫的页面解析能力，支持动态渲染页面（如使用 Selenium 或 Playwright）。这些方向将是我们后续学习与探索的重点。

总而言之，通过本次实验，我们不仅深入理解了信息检索系统的核心理论与关键技术，也在工程实践中锻炼了系统设计、算法实现和性能评价的综合能力，对信息检索领域有了更为全面和深刻的认识。

参考文献

- [1] Manning C D, Raghavan P, Schütze H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 2009, 3(4): 333–389.
- [3] Salton G, Wong A, Yang C S. A Vector Space Model for Automatic Indexing. *Communications of the ACM*, 1975, 18(11): 613–620.
- [4] jieba 中文分词. <https://github.com/fxsjy/jieba>.
- [5] Flask Web Framework. <https://flask.palletsprojects.com/>.
- [6] Baeza-Yates R, Ribeiro-Neto B. *Modern Information Retrieval: The Concepts and Technology behind Search*. 2nd ed. Addison-Wesley, 2011.
- [7] Croft W B, Metzler D, Strohman T. *Search Engines: Information Retrieval in Practice*. Addison-Wesley, 2010.